Assignment No. 03

- **TITLE –** Implement basic array operations such as insert, delete, and merge, while simulating a 2D array using a single-dimensional array to optimize space (using both row-major and column-major formats). Use pointer arithmetic to print memory addresses of elements, helping you understand how arrays are stored in memory.

- **THEORY –**

1. Arrays in C++

An **array** is a collection of elements of the same data type stored in **contiguous memory locations**. It allows **random access** to elements using indices.

Basic Array Operations:

- **Insertion** – Add a new element at a given index.
- **Deletion** – Remove an element (shift or set to 0).
- **Traversal** – Access and display all elements sequentially.
- **Searching** – Find an element's position in the array.
- **Merging** – Combine two arrays into one larger array.

Arrays provide **O(1) access time** but **O(n) insertion/deletion** in general (since elements may need shifting).

2. 2D Arrays

A **2D array** is essentially an array of arrays. It is used to represent tabular data (like matrices). In memory, however, a 2D array is stored **linearly**.

- For int arr[3][3], although it looks like a grid, elements are stored in **contiguous memory** as a 1D sequence.

3. Representing a 2D Array Using a 1D Array

Instead of directly declaring arr[rows][cols], we can simulate it using a **single-dimensional array** with rows * cols elements.

We need a **mapping function** to calculate the index in the 1D array from a pair (i, j):

1. **Row-Major Order**
   - Rows are stored one after another.
   - Formula:
     `index=i×cols+jindex = i \times cols + jindex=i×cols+j`

   Example: In a 3×3 array, element (1,2) (row=1, col=2) is stored at index 1*3 + 2 = 5.
2. **Column-Major Order**
   - Columns are stored one after another.
   - Formula:
     `index=j×rows+iindex = j \times rows + iindex=j×rows+i`

   Example: In a 3×3 array, element (1,2) is stored at index 2*3 + 1 = 7.

This mapping allows us to **simulate 2D arrays using a 1D array**.

4. Row-Major vs Column-Major Layouts

- **Row-Major**
  Elements of the same row are stored consecutively in memory.
- Row-Major (3x3):
- [a00 a01 a02 a10 a11 a12 a20 a21 a22]
- **Column-Major**
  Elements of the same column are stored consecutively.

- Column-Major (3x3):
- [a00 a10 a20 a01 a11 a21 a02 a12 a22]

Understanding both layouts is important for **performance optimization** and **memory addressing**.

5. Pointer Arithmetic and Memory Addresses

In C++, arrays are closely tied to pointers.

- The **name of the array** is a pointer to the first element.
- The **address of element (i, j)** can be computed as:

For row-major:

Address(i,j)=Base Address+(i×cols+j)×Size of data type

For column-major:

Address(i,j)=Base Address+(j×rows+i)×Size of data type

➢ ALGORITHM –

1. Create Array2D class with members: dynamic array data, dimensions rows, cols, and size.
2. In constructor, allocate **rows*cols** memory and initialize elements to 0.
3. Define index mapping:
   - **Row-major** → i*cols + j
   - **Column-major** → j*rows + i
4. Implement operations:
   - **Insert** (row/col major) → place value at computed index.
   - **Delete** → set element to 0.
   - **Print** → display elements with memory addresses using pointer arithmetic.
5. Input elements row-wise from user.
6. Implement **merge()** → create larger array, copy first array, then append second array.
7. In main():
   - Take dimensions, create object, input elements.
   - Print in row-major and column-major layouts.
   - Delete an element and reprint.
   - Merge with another array and display result.
8. Use destructor to free memory.

➢ EXAMPLE –

1. Example Of Implementation

```
Enter number of rows and columns respectively:
3 3
Enter elements for a 3x3 array (row-wise):
Element (0,0): 1
Element (0,1): 2
Element (0,2): 3
Element (1,0): 4
Element (1,1): 5
Element (1,2): 6
Element (2,0): 7
Element (2,1): 8
Element (2,2): 9
```

Row-Major Layout:
1(0x600003600) 2(0x600003604) 3(0x600003608)
4(0x60000360C) 5(0x600003610) 6(0x600003614)
7(0x600003618) 8(0x60000361C) 9(0x600003620)

Column-Major Layout:
1(0x600003600) 4(0x60000360C) 7(0x600003618)
2(0x600003604) 5(0x600003610) 8(0x60000361C)
3(0x600003608) 6(0x600003614) 9(0x600003620)

Deleting element in row-major...
Row-Major Layout:
1(0x600003600) 2(0x600003604) 3(0x600003608)
0(0x60000360C) 5(0x600003610) 6(0x600003614)
7(0x600003618) 8(0x60000361C) 9(0x600003620)

Merging two arrays:
1(0x600005600) 2(0x600005604) 3(0x600005608)
0(0x60000560C) 5(0x600005610) 6(0x600005614)
7(0x600005618) 8(0x60000561C) 9(0x600005620)
100(0x600005624) 0(0x600005628) 0(0x60000562C)
0(0x600005630) 200(0x600005634) 0(0x600005638)

- **CODE –**

```
#include <iostream>
using namespace std;

class Array2D {
    int *data;
    int rows, cols, size;
public:
    Array2D(int r, int c){
        rows=r;
        cols=c;
        size=rows*cols;
        data=new int[size];
        for (int i=0; i<size; i++){
            data[i]=0;
        }
    }

    int rowMajorIndex(int i, int j){
        return i*cols+j;
    }

    int colMajorIndex(int i, int j){
        return j*rows+i;
    }

    void insertRowMajor(int i, int j, int val){
        int index=rowMajorIndex(i, j);
        data[index]=val;
    }
```

```
    void insertColMajor(int i, int j, int val){
        int index=colMajorIndex(i, j);
        data[index]=val;
    }

    void deleteRowMajor(int i, int j){
        int index=rowMajorIndex(i, j);
        data[index]=0;
    }

    void printRowMajor(){
        cout <<endl<<"Row-Major Layout:"<<endl;
        for (int i=0; i<rows; i++) {
            for (int j=0; j<cols; j++) {
                int idx=rowMajorIndex(i, j);
                cout<<data[idx]<<"("<<(data + idx)<<")\t";
            }
            cout<<endl;
        }
    }

    void printColMajor(){
        cout<<"\nColumn-Major Layout:"<<endl;
        for (int i=0; i<rows; i++) {
            for (int j=0; j<cols; j++) {
                int idx = colMajorIndex(i, j);
                cout<<data[idx]<<"("<<(data + idx)<<")\t";
            }
            cout<<endl;
        }
    }

    void inputElementsRowMajor(){
        cout << "Enter elements for a " << rows << "x" << cols << " array (row-wise):"
<< endl;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                int val;
                cout << "Element (" << i << "," << j << "): ";
                cin >> val;
                insertRowMajor(i, j, val);
            }
        }
    }

    static Array2D merge(Array2D &a, Array2D &b){
        int newRows=a.rows +b.rows;
        int newCols=max(a.cols, b.cols);
        Array2D result(newRows, newCols);

        for (int i = 0; i < a.rows; i++) {
            for (int j = 0; j < a.cols; j++) {
                result.insertRowMajor(i, j, a.data[a.rowMajorIndex(i, j)]);
            }
        }
        for (int i = 0; i < b.rows; i++) {
            for (int j = 0; j < b.cols; j++) {
                result.insertRowMajor(i + a.rows, j, b.data[b.rowMajorIndex(i, j)]);
            }
```

```
        }
        return result;
    }
    ~Array2D(){
    delete[] data;
    }
};

int main() {
    cout<<"Enter number of rows and columns respectively: "<<endl;
    int r, c;
    cin>>r>>c;
    Array2D arr(3, 3);
    arr.inputElementsRowMajor();

    arr.printRowMajor();
    arr.printColMajor();

    cout << "\nDeleting element in row-major..." << endl;
    arr.deleteRowMajor(1, 0);

    arr.printRowMajor();

    cout << "\nMerging two arrays:" << endl;
    Array2D arr2(2, 3);
    arr2.insertRowMajor(0, 0, 100);
    arr2.insertRowMajor(1, 1, 200);

    Array2D merged = Array2D::merge(arr, arr2);
    merged.printRowMajor();

    return 0;
}
```

- **OUTPUT –**

```
Enter number of rows and columns respectively:
3 3
Enter elements for a 3x3 array (row-wise):
Element (0,0): 9
Element (0,1): 8
Element (0,2): 7
Element (1,0): 6
Element (1,1): 5
Element (1,2): 4
Element (2,0): 3
Element (2,1): 2
Element (2,2): 1
```

```
Row-Major Layout:
9(0x10b1d60)    8(0x10b1d64)    7(0x10b1d68)
6(0x10b1d6c)    5(0x10b1d70)    4(0x10b1d74)
3(0x10b1d78)    2(0x10b1d7c)    1(0x10b1d80)

Column-Major Layout:
9(0x10b1d60)    6(0x10b1d6c)    3(0x10b1d78)
8(0x10b1d64)    5(0x10b1d70)    2(0x10b1d7c)
7(0x10b1d68)    4(0x10b1d74)    1(0x10b1d80)

Deleting element in row-major...

Row-Major Layout:
9(0x10b1d60)    8(0x10b1d64)    7(0x10b1d68)
0(0x10b1d6c)    5(0x10b1d70)    4(0x10b1d74)
3(0x10b1d78)    2(0x10b1d7c)    1(0x10b1d80)

Merging two arrays:

Row-Major Layout:
9(0x10b1e08)    8(0x10b1e0c)    7(0x10b1e10)
0(0x10b1e14)    5(0x10b1e18)    4(0x10b1e1c)
3(0x10b1e20)    2(0x10b1e24)    1(0x10b1e28)
100(0x10b1e2c)  0(0x10b1e30)    0(0x10b1e34)
0(0x10b1e38)    200(0x10b1e3c)  0(0x10b1e40)
```

- **CONCLUSION –**

  In this practical, we have observed that Arrays can be efficiently simulated using a single-dimensional array with row-major and column-major mappings. This implementation demonstrates basic operations, memory layout, and pointer arithmetic for better understanding of array storage.